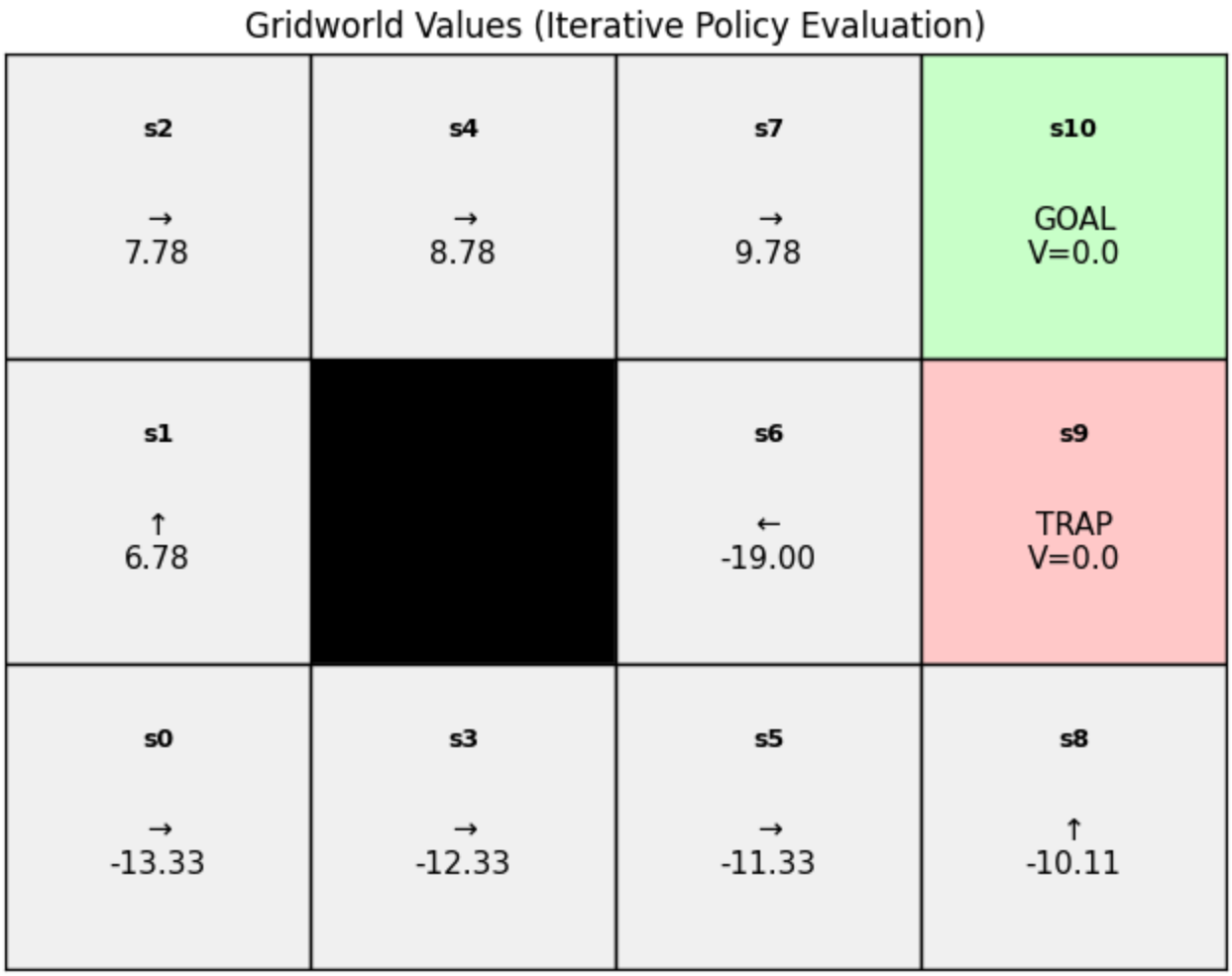
Assignment 1

Q1

(a) State values

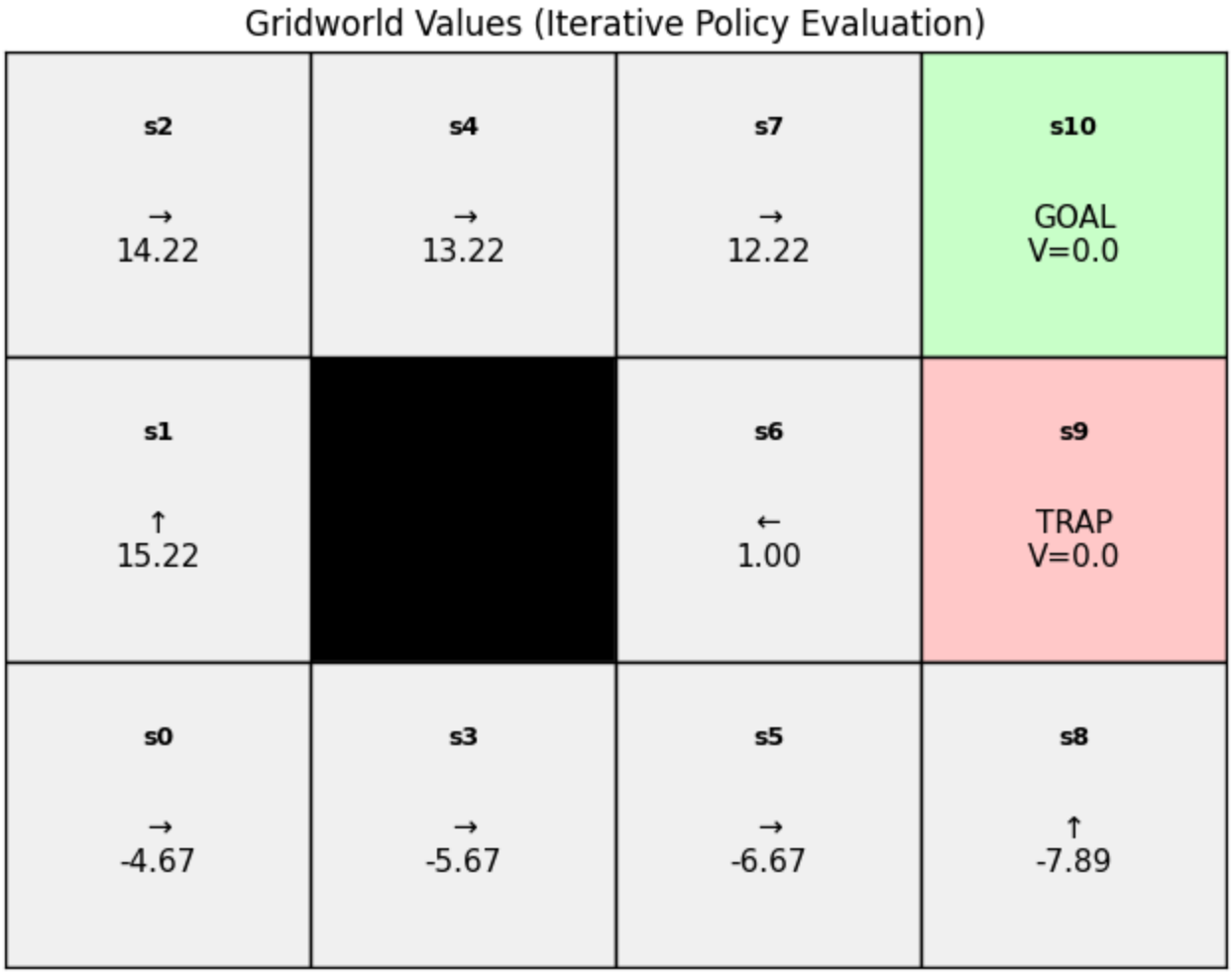


(b) Improvement

- S6 can point upwards towards S7 (intuitively it currently points into the wall and with the uncertainty it has a 10% chance of going straight into the trap state)
- S5 can point upwards to S6 after we change S6 since currently it just leads to the trap state
- Similarly S8 should point at S5 for the same reason

(c)

Result from increasing all reward values by **2**
i.e Goal reward: 12
Trap reward: -8
Step reward: +1



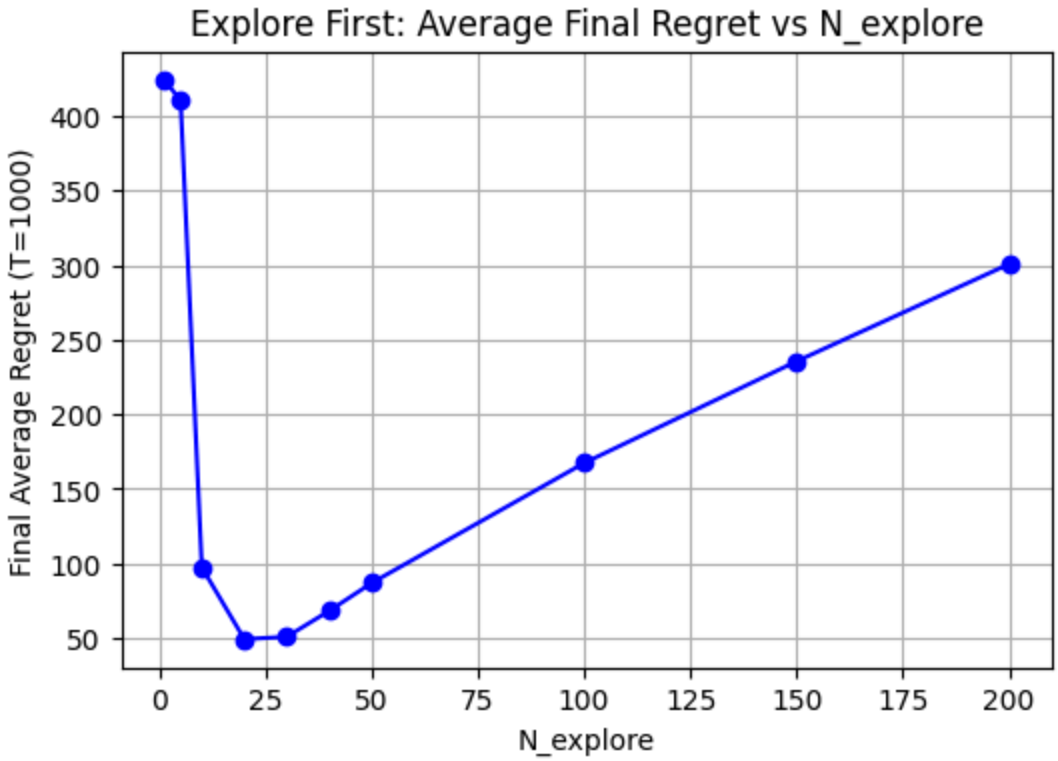
So it seems like each state's value increases by k *number_of_steps in the episode from that state. This means that states further away from the goal get a bigger increase to their value. Here since* k* is large enough to flip the signs for our step reward, it impacts the learning by a lot

In the gridworld problem, if the step reward becomes positive the agent may learn to just keep looping somewhere to get maximum reward, if we take the episode to end once it reaches the goal, it will get a higher return by looping around than by reaching the goal.

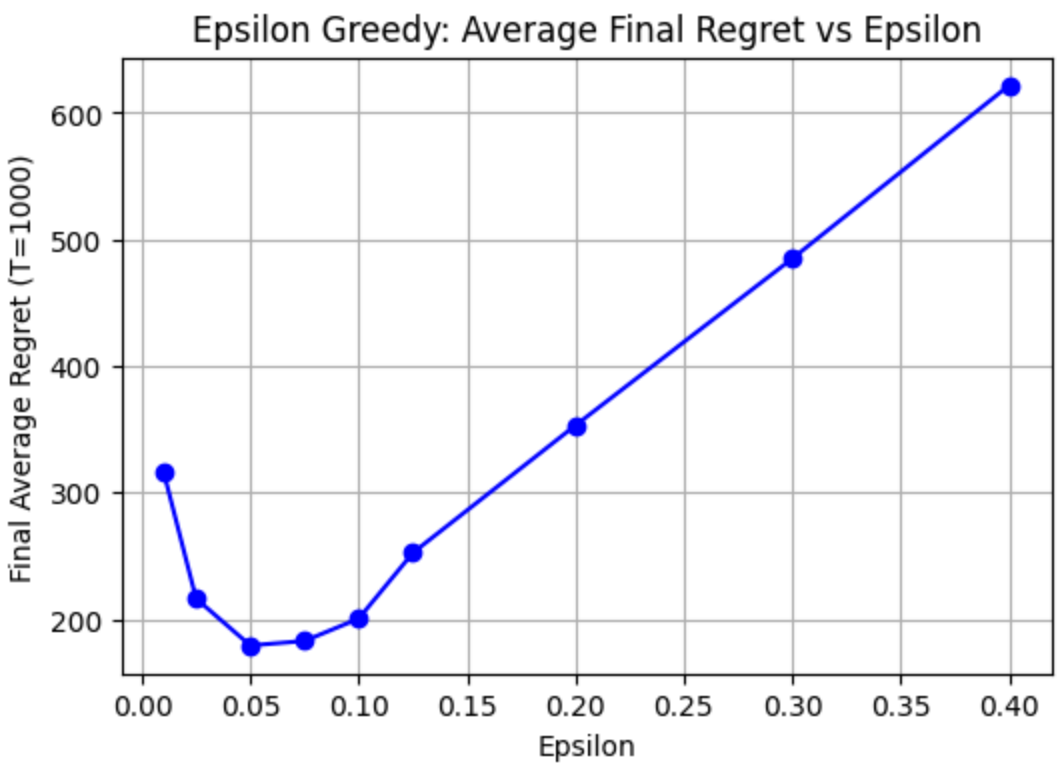
Q2

(a) Plots for each algorithm

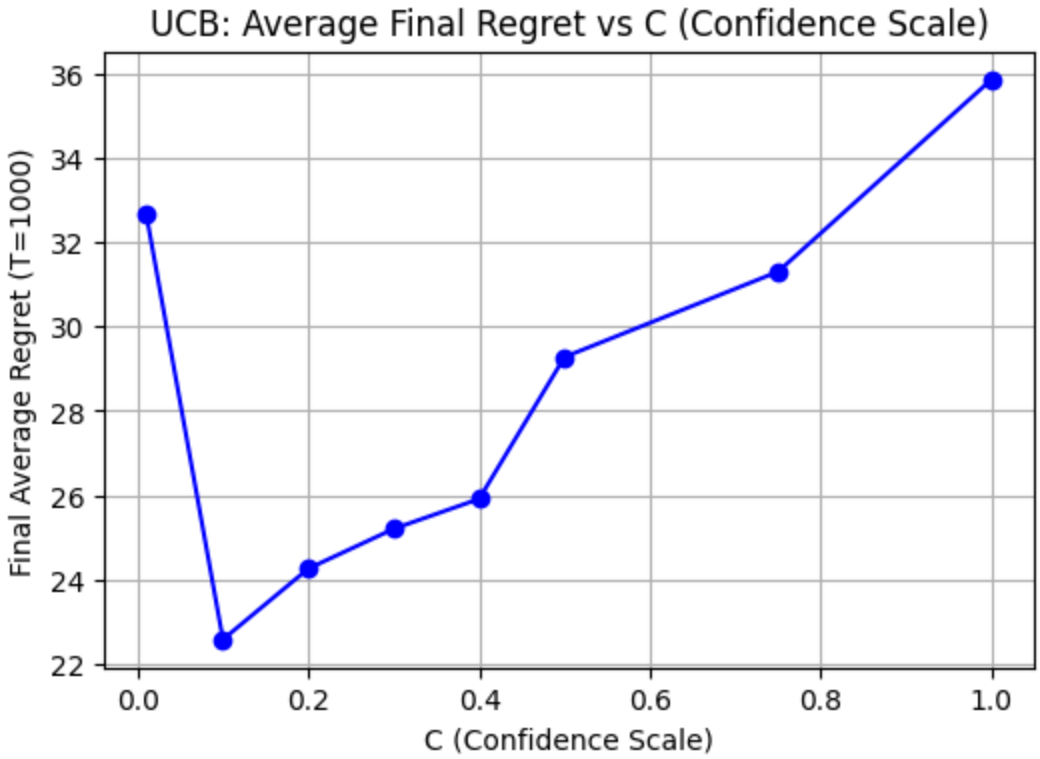
Explore then Commit



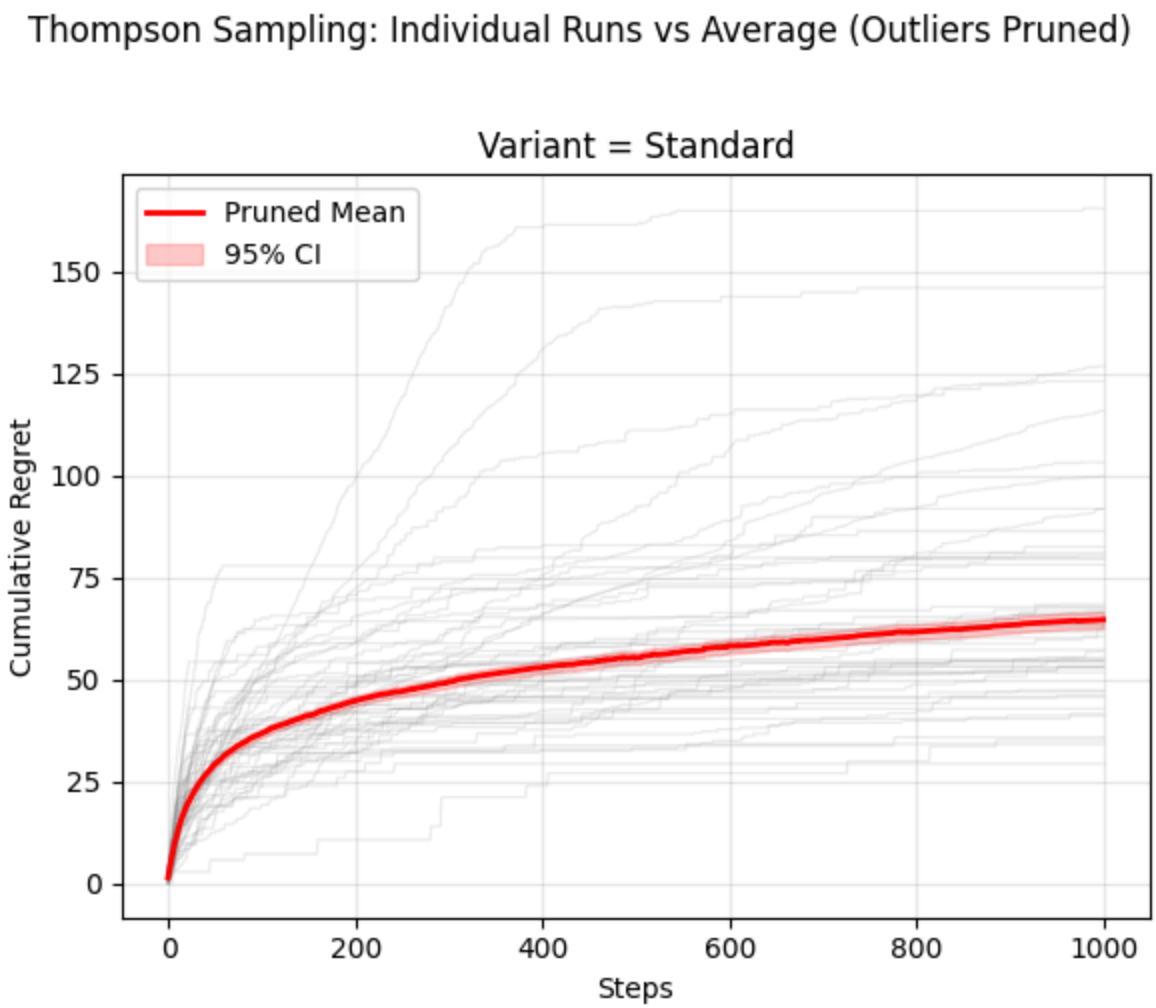
Epsilon Greedy



UCB


UCB: Average Final Regret vs C (Confidence Scale)
Final Average Regret (T=1000)
C (Confidence Scale)
36
34
32
30
28
26
24
22
0.0
0.2
0.4
0.6
0.8
1.0

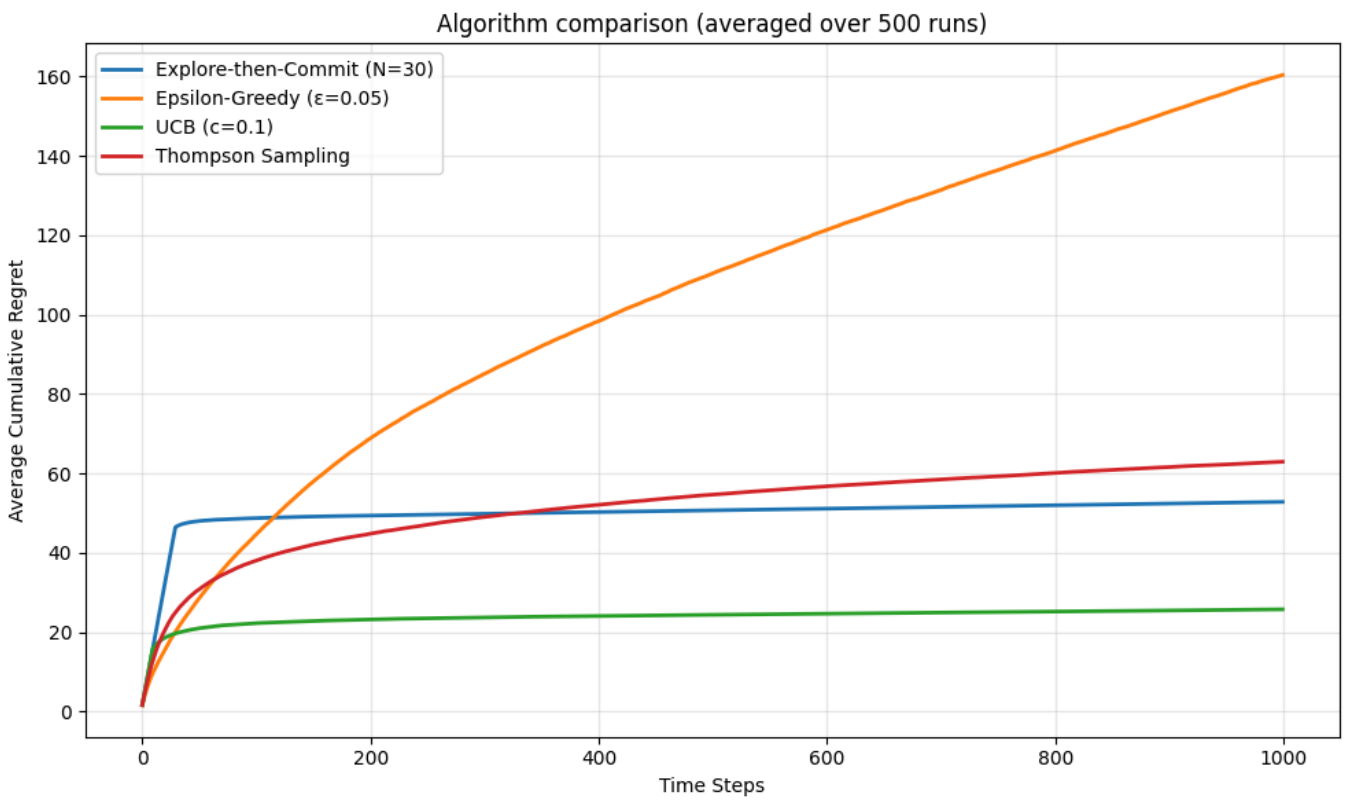
Thompson


Thompson Sampling: Individual Runs vs Average (Outliers Pruned)
Variant = Standard
Pruned Mean
95% CI
Cumulative Regret
Steps
150
125
100
75
50
25
0
0
200
400
600
800
1000

(b) Explanation on performance and optimal parameters

- From the above graphs, we can see that UCB performs much better than the other algorithms by a good margin. Then we can see its pretty close between Explore then commit and Thompson, and finally Epsilon greedy is pretty bad.
- I ran each algorithm 500 times, I chose this because for some instances (like UCB at 0.01, Epsilon greedy at 0.01) there are a lot of "outliers" which tend to skew the average, and after pruning those outliers there were too little samples in those instances to have a meaningful comparison so decided to run a large number of iterations.
- Each of those algorithms gave a nice elbow graph from which it was possible to pick the optimal parameter directly.

(c) Plot comparing all algos



Optimal Parameters Chosen:
Explore then commit: 40
Epsilon: 0.05
UCB: 0.1

Q3 AlphaGo

(a)

- Primarily alphago used CNNs to evaluate the board state rather than relying on hardcoded states. This allowed the agent to find patterns and features that humans potentially couldn't see or model in the monte carlo search tree. It also allowed the model to prune the search tree and evaluate only likely moves.
- Also to evaluate the current state of the board, previous methods needed full evaluation of the monte carlo tree, alphago again used a CNN to evaluate much faster, and also accounted for variations not present in the tree.
- They also made alphago play against itself millions of times so it can learn a lot more than it would otherwise

(b)

- Since the model relies a lot on human data at the start, it could bias it into learning bad human patterns or missing certain good moves.
- Also the architecture is not unified, it has a lot of moving parts that need to be properly tuned so they can work together.

(c)

AlphaGo combined the concepts of search trees, neural networks and reinforcement learning to give us something great. This is even more evident in current research on LLMs + RL both the components complement each other and make each other better.